Subject:	OOP-JAVA
Name:	Yug Gandhi
ID No.:	25AIML013

Practical-11

GIT REPOSITRY LINK : https://github.com/25aiml013-sudo/JAVA-PRACTICALS.git

Part A

PROBLEM-1 :

Save/load: make a small class Serializable, save an array of its objects to a file with ObjectOutputStream, then read it back into fresh objects and confirm the values survived. Mark one field transient and show it is not saved.

CODE :

```
import java.io.*;

class Student implements Serializable {
    private static final long serialVersionUID = 1L;

    int id;
    String name;
    transient String password;

    Student(int id, String name, String password) {
        this.id = id;
```

```
        this.name = name;
        this.password = password;
    }
}

public class SerializationDemo {
    public static void main(String[] args) {
        Student[] students = {
            new Student(1, "Rahul", "abc123"),
            new Student(2, "Amit", "xyz456")
        };

        try {
            ObjectOutputStream out = new ObjectOutputStream(
                new FileOutputStream("students.dat")
            );
            out.writeObject(students);
            out.close();

            ObjectInputStream in = new ObjectInputStream(
                new FileInputStream("students.dat")
            );

            Student[] loaded = (Student[]) in.readObject();
            in.close();

            for (Student s : loaded) {
                System.out.println("ID: " + s.id);
```

```
            System.out.println("Name: " + s.name);
            System.out.println("Password: " + s.password);
            System.out.println();
        }
    } catch (IOException | ClassNotFoundException e) {
        System.out.println(e.getMessage());
    }
  }
}
```

OUTPUT :

```
cd "/Users/yuggandhi/Desktop/SEM-3/JAVA/JAVA/" && javac Serializat
(base) yuggandhi@Yugs-MacBook-Pro JAVA % cd "/Users/yuggandhi/Desk
a && java SerializationDemo
ID: 1
Name: Rahul
Password: null

ID: 2
Name: Amit
Password: null
```

PROBLEM-2 :

Log analyzer: create a few text files, then use Path/Files to read each line by line and count the total lines and how many contain a keyword across all files; also print each file's size from its attributes.

CODE :

```
import java.io.IOException;
import java.nio.file.*;
```

```
import java.nio.file.attribute.BasicFileAttributes;
import java.util.*;

public class LogAnalyzer {
   public static void main(String[] args) throws IOException {
      Path folder = Paths.get("logs");

      Files.createDirectories(folder);

      Files.write(folder.resolve("log1.txt"),
         Arrays.asList("Application started", "Error found", "User logged in"));

      Files.write(folder.resolve("log2.txt"),
         Arrays.asList("System running", "Error occurred", "Application closed"));

      int totalLines = 0;
      int keywordCount = 0;
      String keyword = "error";

      try (DirectoryStream<Path> files =
            Files.newDirectoryStream(folder, "*.txt")) {

         for (Path file : files) {
            int fileLines = 0;

            try (Scanner scanner = new Scanner(file)) {
               while (scanner.hasNextLine()) {
                  String line = scanner.nextLine();
```

```
                totalLines++;
                fileLines++;

                if (line.toLowerCase().contains(keyword)) {
                   keywordCount++;
                }
              }
            }

            BasicFileAttributes attr =
               Files.readAttributes(file, BasicFileAttributes.class);

            System.out.println("File: " + file.getFileName());
            System.out.println("Size: " + attr.size() + " bytes");
            System.out.println("Lines: " + fileLines);
            System.out.println();
          }
        }

        System.out.println("Total lines: " + totalLines);
        System.out.println("Lines containing '" + keyword +
                    "': " + keywordCount);
    }
}
```

OUTPUT :

```
(base) yuggandhi@Yugs-MacBook-Pro JAVA % cd "/Users/yuggandh
ava LogAnalyzer
File: log2.txt
Size: 49 bytes
Lines: 3

File: log1.txt
Size: 47 bytes
Lines: 3

Total lines: 6
Lines containing 'error': 2
(base) yuggandhi@Yugs-MacBook-Pro JAVA %
```

PROBLEM-3 :

Walk a folder tree with NIO and produce a report of the files it contains (name, size, modified time), writing the summary to a file; decide what to include.

CODE :

```
import java.io.*;
import java.nio.file.*;
import java.nio.file.attribute.BasicFileAttributes;

public class FolderReport {
    public static void main(String[] args) throws IOException {
        Path folder = Paths.get("logs");
        Path report = Paths.get("folder-report.txt");

        try (BufferedWriter writer = Files.newBufferedWriter(report)) {
            writer.write("FOLDER TREE REPORT");
            writer.newLine();
            writer.write("==================");
            writer.newLine();

            Files.walkFileTree(folder, new SimpleFileVisitor<Path>() {
                @Override
```

```
            public FileVisitResult visitFile(
                Path file, BasicFileAttributes attrs)
                throws IOException {

              writer.write("Name: " + file.getFileName());
              writer.newLine();

              writer.write("Size: " + attrs.size() + " bytes");
              writer.newLine();

              writer.write("Modified: " +
                attrs.lastModifiedTime());
              writer.newLine();

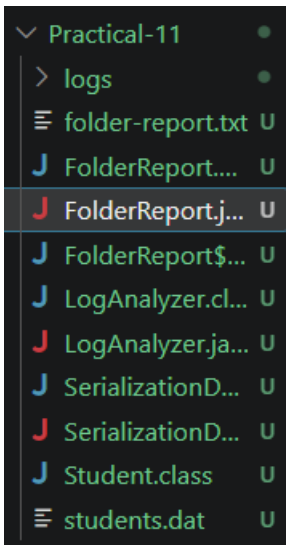
              writer.write("------------------");
              writer.newLine();

              return FileVisitResult.CONTINUE;
            }
          });
      }

      System.out.println("Report created: " + report.toAbsolutePath());
    }
}
```

OUTPUT :

```
(base) yuggandhi@Yugs-MacBook-Pro JAVA % cd "/Users/yuggandhi/Desktop/SEM-3/JAVA/JAVA/" && javac FolderReport.jav
java FolderReport
Report created: /Users/yuggandhi/Desktop/SEM-3/JAVA/JAVA/folder-report.txt
(base) yuggandhi@Yugs-MacBook-Pro JAVA %
```



Key Questions :

1. What is Serializable, and why add serialVersionUID?
Serializable allows an object to be saved as bytes. serialVersionUID identifies the class version during serialization and deserialization.

2. Difference between byte stream and character stream?

- Byte stream: Handles binary data, such as images (InputStream, OutputStream).
- Character stream: Handles text data (Reader, Writer).

3. Difference between NIO Path/Files and the old File class? How is a directory stream iterated?
Path represents a file path, and Files provides advanced file operations. The old File class offers basic file handling. A DirectoryStream is iterated using a for loop to access directory entries.